
Adaptive Interaction Graphs for Particle Simulation

Aiden Zhou
Yale University
aiden.zhou@yale.edu

Abstract

Learned particle simulators based on graph neural networks achieve strong one-step accuracy, but errors compound over long horizons. An underexplored variable is the interaction graph: existing methods fix its topology via k -nearest neighbors or a static radius rule, regardless of local model confidence. We propose making this graph adaptive: a per-particle variance head, trained jointly with the acceleration head under a heteroscedastic Gaussian NLL loss, drives a trajectory in which high-uncertainty particles receive an expanded neighborhood. This is done at no extra inference cost by using the previous step’s uncertainty estimate. A key discovery is that the variance head learns a meaningful notion of uncertainty. Regions of high variance correspond to splash zones, collisions, and free surfaces. When this signal drives graph topology, the resulting ADAPTGNS simulator achieves a strict Pareto improvement on WATERDROP and a modest gain on SAND. Given the model’s stronger performance on WATERDROP, we hypothesize that adaptive graphs are most useful when physical complexity is concentrated in space. Our code is available at <https://github.com/aidenzhou8/AdaptGNS>.

1 Introduction

Problem statement. Learned simulators based on graph neural networks (GNNs) have shown strong one-step accuracy on particle dynamics tasks, but are hurt by compounding error over many steps Sanchez-Gonzalez et al. [2020]. A key architectural choice in such systems is the **interaction graph**, which decides what particles exchange messages at each time step. Existing methods fix this topology using static geometric heuristics, such as a radius rule Sanchez-Gonzalez et al. [2020] or a learned edge mask Kipf et al. [2018], regardless of the model’s local confidence.

We propose ADAPTGNS, an **uncertainty-adaptive interaction graph** that selectively expands edges per timestep based on the simulator’s predicted uncertainty. By providing greater relational capacity where the model is least confident — and leaving the default sparse connectivity in place everywhere else — ADAPTGNS aims to make graph network simulators (GNSs) more stable, but also to preserve the speed and efficiency that motivate using a learned simulator in the first place.

Motivation and significance. Long-horizon physical simulation is key to a wide range of subjects, such as robotics, molecular dynamics, and world models. Learned simulators can provide much faster inference than classical solvers, with reported speedups of up to $5,000\times$ over the Material Point Method (MPM) on granular flow simulations Kumar and Vantassel [2023]. However, this benefit is lost if errors accumulate over many steps. Enhancing consistency without increasing average compute per step would make learned simulators more viable to replace classical solvers.

Learning objective. Our task is per-particle acceleration prediction at each timestep, with the predicted values integrated forward to produce a complete simulation trajectory. Errors at each step compound over many steps, and our goal is to reduce this error growth relative to baseline models. A secondary objective trains a **per-particle uncertainty head** — a scalar variance estimate σ_i —

that drives the adaptive graph policy at each step. We report two metrics throughout: **MSE** for standardized accuracy between datasets and models and **average edges per step** for efficiency. Note that each processor layer of a GNS does $O(|E|)$ work, so compute scales with edge count.

Summary of results. ADAPTGNS achieves a strict Pareto improvement on WATERDROP (20% fewer edges and lower MSE at step 200 (MSE@200) than fixed $k = 5$), but only a modest gain on SAND (12% lower MSE@200 than fixed $k = 5$, using 10% fewer edges than $k = 10$).¹ The story becomes clearer once we ablate for the choice of loss function: compared to a fixed GNS **trained on NLL loss**, the adaptive policy once more achieves a strong gain (21% fewer edges and 8% lower MSE@200). We also report the optimized hyperparameters found by grid sweep on SAND and WATERDROP (σ -percentile = 70, radius factor = 1.267) and several negative results.

2 Dataset

Description. We use two distinct particle simulation datasets. SAND is taken from Kumar and Vantassel [2023], hosted on DesignSafe-CI (DOI: 10.17603/ds2-0phb-dg64), and generated with the Taichi-MPM solver. WATERDROP is taken from Sanchez-Gonzalez et al. [2020] and generated with a Smoothed Particle Hydrodynamics (SPH) solver. Note that WATERDROP trajectories are given as TFRecords, so we provide a script to convert the dataset to a standard .npz format. These two materials were chosen to evaluate adaptive graphs across both fluid and granular flow regimes.

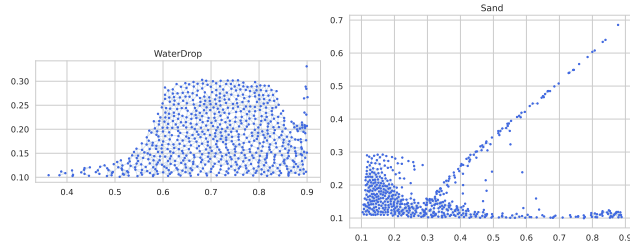


Figure 1: Interaction graph snapshots at Step 100 for WATERDROP (left) and SAND (right).

Graph statistics. The edges of a standard interaction graph are constructed at each step between particles separated by at most $r = 0.015$. Table 1 reports graph statistics taken at Step 100, averaged across trajectories. Both datasets produce sparse graphs with moderate average degree. Notably, both datasets have an average clustering coefficient of roughly 0.50, indicating that roughly half of each particle’s neighbor-pairs are also mutual neighbors.

Table 1: Graph statistics at Step 100

Dataset	Trajs	Nodes	Edges	Density	Avg Degree	Avg Clustering
WATERDROP	30	567 ± 200	1550 ± 583	1.08×10^{-2}	5.42 ± 0.22	0.494 ± 0.036
SAND	30	1270 ± 503	4324 ± 1864	7.35×10^{-3}	6.76 ± 1.01	0.500 ± 0.034

The temporal evolution of graph statistics reveals varying dynamics: in WATERDROP, edge count and average degree rise as the droplet spreads and then taper as particles disperse, whereas in SAND the graph undergoes a sharp slump around Step 50 before recovering to a stable plateau.

Feature analysis. **Node features** are the last 5 velocity vectors, concatenated with a particle type indicator. **Edge features** are the relative displacement vector and distance between particles. **Labels** are per-particle acceleration at the next step. The acceleration magnitude distribution is heavily right-skewed in both datasets. The vast majority of particles have near-zero acceleration at any given step, with rare high-magnitude events at impact zones.

¹ k is roughly the average number of neighbors per particle at the given connectivity radius r . We use this easy notation throughout, but be aware that r is fixed; neighbor count varies with local particle density.

Data splits. We use those provided alongside each dataset. In particular, note that trajectories are independent and data is split at the trajectory level. For both SAND and WATERDROP, we use the complete training set and evaluate performance on a fixed subset of 30 test trajectories. Hyperparameter tuning was conducted on the validation set only.

Table 2: Data splits per dataset

Dataset	Train trajectories	Validation trajectories	Test trajectories	Steps / trajectory
SAND	1,000	100	30	320
WATERDROP	1,000	30	30	1,000

3 Related Work

The idea of representing physical systems as graphs was introduced by Battaglia et al. [2016], who developed the relational inductive bias that subsequent work extends upon. Sanchez-Gonzalez et al. [2020] scaled this paradigm to large particle systems via radius-based interaction graphs and noise-augmented training. This work forms the primary baseline and codebase for our project. A central restriction shared by both works is that the interaction graph is fixed by geometric rules, which do not regard local dynamics. Topping et al. [2022] provide the theoretical grounding for why we care: data from distant nodes is sharply compressed through bottleneck edges (over-squashing), and this worsens over long periods to cause compounding error.

The most straightforward try to learn the graph structure itself came from Kipf et al. [2018], whose Neural Relational Inference (NRI) derives a discrete interaction graph via a VAE encoder over particle histories. However, NRI produces a graph that is fixed at inference, and incurs significant overhead. More recently, AdaMeshNet Seo et al. [2025] proposes per-step adaptation for mesh-based GNNs using geometric heuristics, outperforming static baselines. Our work diverges from both: we target point-cloud particle systems rather than meshes, use no separate encoder, and use the model’s own predicted uncertainty σ_i as the adaptation signal. This leads to a lightweight, general architecture that adapts at every step as a byproduct of the dynamics head itself.

4 Methodology

Variance head. We extend a standard GNS with a **variance prediction head**: alongside the acceleration output, the network predicts a scalar uncertainty σ_i per particle. This is done via a two-layer MLP on the final node embeddings, using a softplus activation to enforce positivity:

$$\sigma_i^2 = \text{softplus}(f_\theta(h_i)).$$

This change adds fewer than 20,000 parameters relative to the standard GNS, so the architectural overhead is inconsequential. See `VarianceHead` in `adaptive-gns/gns/graph_network.py`.

Heteroscedastic NLL loss. Both acceleration and variance heads are trained jointly under a Gaussian NLL loss:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \frac{(\hat{a}_i - a_i)^2}{2\sigma_i^2} + \frac{1}{2} \log \sigma_i^2. \quad (1)$$

In contrast to MSE, which provides no gradient to σ_i , NLL ensures that the model is rewarded for concentrating uncertainty where errors are large and punished for inflating it elsewhere.

Adaptive graph construction loop. Our core contribution is an inference-time loop that uses predicted uncertainty to modify the graph topology. A naive implementation would run a GNN forward pass, extract σ_i , rebuild the graph, and then run a second forward pass to compute the final acceleration. However, this doubles inference cost. Instead, we use a **single-pass lagged- σ** design. At step t , the graph is constructed using the previous step’s uncertainty estimates $\sigma_i^{(t-1)}$. The forward pass at step t then produces both the next position and a fresh uncertainty estimate $\sigma_i^{(t)}$ for step $t + 1$. Concretely, our adaptive mechanism consists of three operations:

1. Construct the standard interaction graph (with radius r).
2. Identify particles whose σ_i is above a percentile threshold τ .
3. Add extra edges from a larger radius $r_{\text{large}} = \text{radius_factor} \cdot r$ for edges incident to one or more high-uncertainty particles.

See `predict_positions_adaptive` and its helper functions in `adaptive-gns/gns/learned_simulator.py` for more information and implementation details.

5 Experiments and Results

5.1 Experimental setup.

Hyperparameters. The adaptive policy has two primary hyperparameters: the uncertainty threshold τ and the radius expansion factor `radius_factor`. Particles above the τ -th percentile in predicted variance receive expanded neighborhoods, whereas other particles keep the default connectivity.

We choose `radius_factor` so that the enlarged adaptive radius matches the radius used by the fixed $k = 10$ baseline. As the base $k = 5$ graph uses $r = 0.015$ and the $k = 10$ case corresponds to roughly $r = 0.019$, we set `radius_factor` = $\frac{0.019}{0.015} \approx 1.267$. This gives high- σ particles in the adaptive model exactly the same connectivity as the strongest fixed-graph baseline. We verify this choice by tuning τ and `radius_factor` jointly via a grid search over $\tau \in \{65, 70, 75, 80, 90\}$ and `radius_factor` $\in \{1.2, 1.267, 1.5, 2\}$. Indeed, we found that $\tau = 70$, `radius_factor` = 1.267 is the optimal configuration on the validation set.

Baselines and Ablations. We compare ADAPTGNS to five models on both WATERDROP and SAND. The naive comparison is to a **per-particle MLP**, which measures the value of relational inductive bias. Far more reasonable models are a **fixed-graph GNS with $k = 5$** (or $r = 0.015$), which serves as the primary comparison, and a **fixed-graph GNS with $k = 10$** (or $r = 0.019$), which gives an upper bound on the benefit of denser interaction graphs.

We also conduct two ablation studies. **GNS (k -mid)** uses an intermediate radius ($r = 0.0176$) matched to ADAPTGNS’s mean edge count, isolating whether our gains come from adaptive topology or just from having more edges on average.² **ADAPTGNS fixed-graph uses NLL** but with fixed $r = 0.015$, isolating the contribution of the adaptive loop from the changed loss function.

Computational details. Everything was run on Yale’s Misha cluster, and scheduled via SLURM jobs in `jobs/`. Each 500,000-step training run takes roughly 6 hours on an Nvidia A100 GPU, and evaluating a model on the test set of WATERDROP or SAND takes at most 30 minutes.

5.2 Quantitative Results

Our first question is whether the variance head reveals physical structure. If σ_i is just noise, a graph policy that depends on it is doubtful to succeed. Fortunately, we come to a positive conclusion.

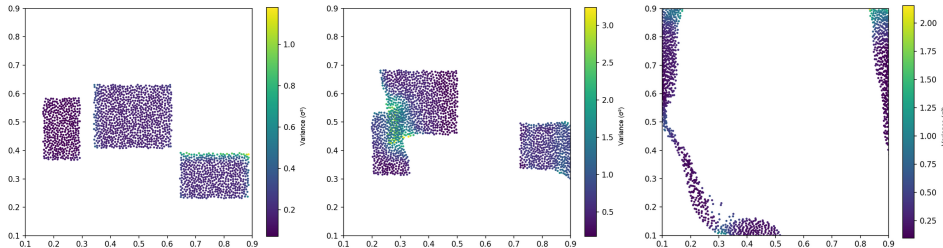


Figure 2: Per-particle uncertainty σ_i at steps 1, 10, and 50 of a SAND trajectory. High- σ particles are concentrated in impact zones, free surfaces, and regions of rapid velocity change.

²Note that this ablation was not conducted for WATERDROP. On SAND, k_{mid} was constructed by interpolating in r^2 between the $k = 5$ and $k = 10$ edge counts to match ADAPTGNS’s budget, resulting in $r \approx 0.0176$. On WATERDROP, ADAPTGNS already uses **fewer** edges than fixed $k = 5$, so no equivalent comparison exists.

Takeaway 1

As shown in Figure 2, the network correctly infers what “uncertainty” means in the context of physical simulation, purely as a byproduct of the learning objective. This is supported by our calculations in Table 3. On SAND, Bins 0–8 are tightly calibrated, with $|\Delta| \leq 0.06$. The total ECE is large only because the model severely underestimates its error on the hardest particles (Bin 9). Results are weaker on WATERDROP, but conform to the same trend. As our adaptive methodology thresholds at a percentile rank, this does not pose a huge obstacle.

Table 3: Per-decile calibration on SAND and WATERDROP.

Bin	SAND			WATERDROP		
	mean SE	$2\hat{\sigma}^2$	$ \Delta $	mean SE	$2\hat{\sigma}^2$	$ \Delta $
0	0.023	0.009	0.014	0.543	0.003	0.540
1	0.025	0.014	0.011	0.540	0.004	0.537
2	0.024	0.018	0.006	0.131	0.009	0.121
3	0.026	0.023	0.003	0.081	0.015	0.066
4	0.033	0.029	0.004	0.208	0.022	0.186
5	0.046	0.038	0.008	0.495	0.032	0.463
6	0.067	0.051	0.016	1.386	0.048	1.339
7	0.141	0.084	0.057	3.167	0.065	3.101
8	0.560	0.195	0.365	5.739	0.094	5.645
9	15.482	1.882	13.599	17.878	0.288	17.590
Total ECE			1.408	2.959		

Next, we evaluate ADAPTGENS on SAND and WATERDROP. See Table 4 for a summary of results and Figure 3 for more nuanced dynamics.

SAND. Comparing ADAPTGENS to its fixed-graph counterpart (using NLL and $r = 0.015$), the adaptive policy **reduces edges by 21% and lowers MSE@200 by 8% (0.0417 vs. 0.0382)**. This is the cleanest comparison: it isolates the value of placing edges purposely rather than uniformly. Second, comparing ADAPTGENS to k_{mid} ($r = 0.0176$) at matched edge counts ($\sim 23,000$ edges each), **the model using MSE wins (0.0355 vs. 0.0382)**. The benefit of MSE as an easier learning objective is made clear by looking at the NLL model at $r = 0.0176$. Although it also uses a larger fixed radius, its MSE@200 is much worse (0.0422).

WATERDROP. The result here is a strong advance: ADAPTGENS **uses 20% fewer edges than $k = 5$ (7,821 vs. 9,741) and also achieves lower MSE@200 (0.0412 vs. 0.0414)**. Even compared to $k = 10$, the tradeoff is 36% fewer edges for a 6% relative MSE@200 increase. In contrast to SAND, the decrease in performance caused by using NLL instead of MSE is close to zero.

Table 4: Results on SAND (314 steps) and WATERDROP (994 steps).

SAND			WATERDROP		
Model	Edges/step	MSE@200	Model	Edges/step	MSE@200
MLP	N/A	0.121	MLP	N/A	0.207
$k = 5$ ($r = 0.015$)	18,682	0.0434	$k = 5$ ($r = 0.015$)	9,741	0.0414
k_{mid} ($r = 0.0176$)	23,498	0.0355	$k = 10$ ($r = 0.019$)	12,212	0.0388
$k = 10$ ($r = 0.019$)	25,516	0.0285	NLL ($r = 0.015$)	7,875	0.0417
NLL ($r = 0.015$)	28,968	0.0417	AdaptGENS	7,821	0.0412
NLL ($r = 0.0176$)	28,768	0.0422			
AdaptGENS	22,958	0.0382			

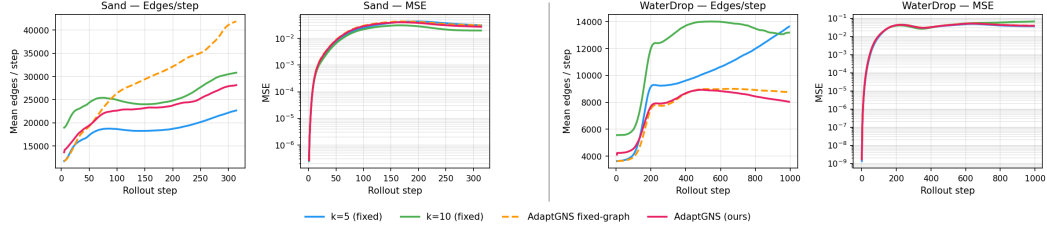


Figure 3: Per-step dynamics (edge count and MSE) on SAND and WATERDROP.

Takeaway 2

ADAPTGNS strictly beats the same NLL model without the adaptive graph mechanism on both datasets. On WATERDROP, it also achieves a strong advance over every fixed-graph comparison. Due to its harder learning objective, ADAPTGNS does not yet defeat the best MSE-trained models on SAND.

5.3 Analysis and Discussion

Hypotheses revisited. To **RQ1** (does adaptive expansion reduce MSE at increasingly long horizons?) and **RQ2** (does the variance head identify physical structure?), **we answer yes!** On both WATERDROP and SAND, we observe a significant gain over the base NLL model when the adaptive mechanism is ablated. As shown in Figure 2 and Table 3, the variance head learns a relevant concept of physical uncertainty. **RQ3** (match $k = 10$ accuracy with fewer edges?) is true on WATERDROP but only partly on SAND, where the k_{mid} ablation shows that a standard GNS (using MSE) at the same edge budget wins on raw accuracy.

Why WATERDROP and not SAND? The performance gap between datasets can be attributed to two physical properties. First, **graph density evolves differently**. Most granular trajectories end up packing particles into an increasingly dense heap. A droplet impact does the opposite, dispersing particles and keeping the graph sparse for the rest of the trajectory. Second, **uncertainty is more heterogeneous in WATERDROP**: a splash creates local complexity but is mostly surrounded by quiet, giving the variance head plenty of contrastive signal. In SAND, nearly every particle undergoes the same dynamic, as was previously observed in our exploratory data analysis.

Edge count is a noisy proxy for compute. The radius graph at step t is constructed from the model’s predicted positions. Hence, a key observation is that **worse models can drift into denser configurations and end up using more edges**. This is shown by our results in Table 4 and Figure 3.

6 Conclusion

In this paper, we explored making the interaction graph of a learned particle simulator **adaptive** — sparse where the model is confident, denser where it is not — without paying the cost of two GNN computations per step. We contribute a **single-pass adaptive mechanism** driven by a **per-particle variance head**, which is trained jointly with the acceleration head under **NLL loss**. We observe that σ_i corresponds to real physical uncertainty, and that ADAPTGNS, driven by this signal, achieves notable results on the WATERDROP and SAND datasets.

This conclusion comes with several caveats. For one, switching from MSE to NLL costs around 30% relative MSE@200 on SAND, and the adaptive mechanism reduces but does not erase this gap. Second, the **single-pass design reacts one step late**. Hence, during the most rapid transitions, σ_i may not reflect the current step’s uncertainty. Lastly, **absolute calibration is poor** on both datasets. This is acceptable in our case, but would impact any downstream task that used the actual variance values. Future work should target a learned next-step uncertainty predictor (solving both lag and calibration concerns) and a learnable per-particle expansion radius. We also intend to test ADAPTGNS on a broader range of physical regimes (e.g. GOOP, MULTIMATERIAL) and on longer-horizon tasks.

References

- Peter W. Battaglia, Razvan Pascanu, Matthew Lai, Danilo Rezende, and Koray Kavukcuoglu. Interaction networks for learning about objects, relations and physics. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL <https://arxiv.org/abs/1612.00222>.
- Thomas Kipf, Ethan Fetaya, Kuan-Chieh Wang, Max Welling, and Richard Zemel. Neural relational inference for interacting systems. In *International Conference on Machine Learning (ICML)*, 2018. URL <https://arxiv.org/abs/1802.04687>.
- Krishna Kumar and Joseph Vantassel. GNS: A generalizable Graph Neural Network-based simulator for particulate and fluid modeling. *Journal of Open Source Software*, 8(88), 2023. doi: 10.21105/joss.05025.
- Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/2002.09405>.
- Sangwoo Seo, Hyunsung Kim, Jiwan Kim, and Chanyoung Park. Adaptive graph rewiring to mitigate over-squashing in mesh-based GNNs for fluid dynamics simulations. *arXiv preprint arXiv:2511.12709*, 2025. URL <https://arxiv.org/abs/2511.12709>.
- Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2111.14522>.